UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
CAMPUS VICTOR LEVIS SASSO
MATERIA: DESARROLLO WEB VII

LABORATORIO: SEGURIDAD EN PHP CON OPENSSL

Firma digital, verificación y encriptación básica, más un ejemplo con Libsodium

Docente: Ing. Irina Fong

Estudiante: Cesar Campbell

Cédula: 8-1041-2239

Compañero(a) de grupo: _____

Período: II Semestre 2026

Fecha: 29 de octubre de 2026

1. Introducción

En este laboratorio trabajamos con la extensión OpenSSL de PHP para proteger datos sensibles. Según OWASP, uno de los errores más comunes en las aplicaciones web son los fallos criptográficos, o sea guardar o mandar contraseñas, tarjetas o datos personales en texto plano. Para evitarlo hay que cuidar dos cosas: la **confidencialidad**, que nadie pueda leer el dato, y la **integridad**, que nadie pueda cambiarlo sin que se note.

Para la confidencialidad usamos cifrado. El cifrado **simétrico** (AES) usa la misma clave para cifrar y descifrar. El **asimétrico** (RSA) usa un par de llaves: la pública se puede compartir y la privada solo la tengo yo. Para la integridad usamos **firmas digitales**: se calcula un hash del mensaje y se cifra con la llave privada, y cualquiera con la llave pública puede comprobar que el mensaje no cambió y que lo firmó el dueño de la llave. Un **certificado X.509** es el documento que une una llave pública con la identidad de alguien.

Ejecutamos los scripts que dio la docente en su repositorio de GitHub, les agregamos formularios para recibir los datos, tomamos capturas de cada prueba y documentamos las funciones. También investigamos Libsodium e hicimos un ejemplo propio con esa librería.

2. Objetivo del laboratorio

Aplicar la extensión OpenSSL y la librería Libsodium en PHP para proteger datos sensibles, entendiendo para qué sirve cada función (firmar, verificar y cifrar) y cómo se relacionan con los lineamientos de OWASP.

3. Desarrollo del laboratorio

3.1 Preparación

Descargamos los archivos del repositorio de la docente (`github.com/Salomon2514/SeguridadOpenSSL`) y armamos la carpeta `LAB_6_OPENSSL` dentro de nuestro proyecto, servida con Apache. Verifiqué que PHP tuviera las dos extensiones activas con `php -m`. Las versiones que usamos fueron:

Componente	Versión
PHP	8.5.12
OpenSSL	3.5.9
Libsodium	1.0.22 (extensión sodium)
Sistema	Linux (Nobara/Fedora) con Apache

Los scripts originales no tenían formulario y traían la ruta fija `C:\OpenSSL-Win64\...`. Pasamos cada uno a una página con formulario y quitamos esa ruta, como indica la solución 1 de la guía. Las páginas comparten un archivo `config.php` con los parámetros de OpenSSL.

3.2 Encriptación simple con OpenSSL

Esta página cifra un texto con AES-256-CBC y lo descifra para comprobar que recupero el original. Primero abrimos el formulario vacío (Captura 1) y después probamos con un dato sensible, un número de tarjeta, y una clave.



Captura 1. Formulario de encriptación vacío.

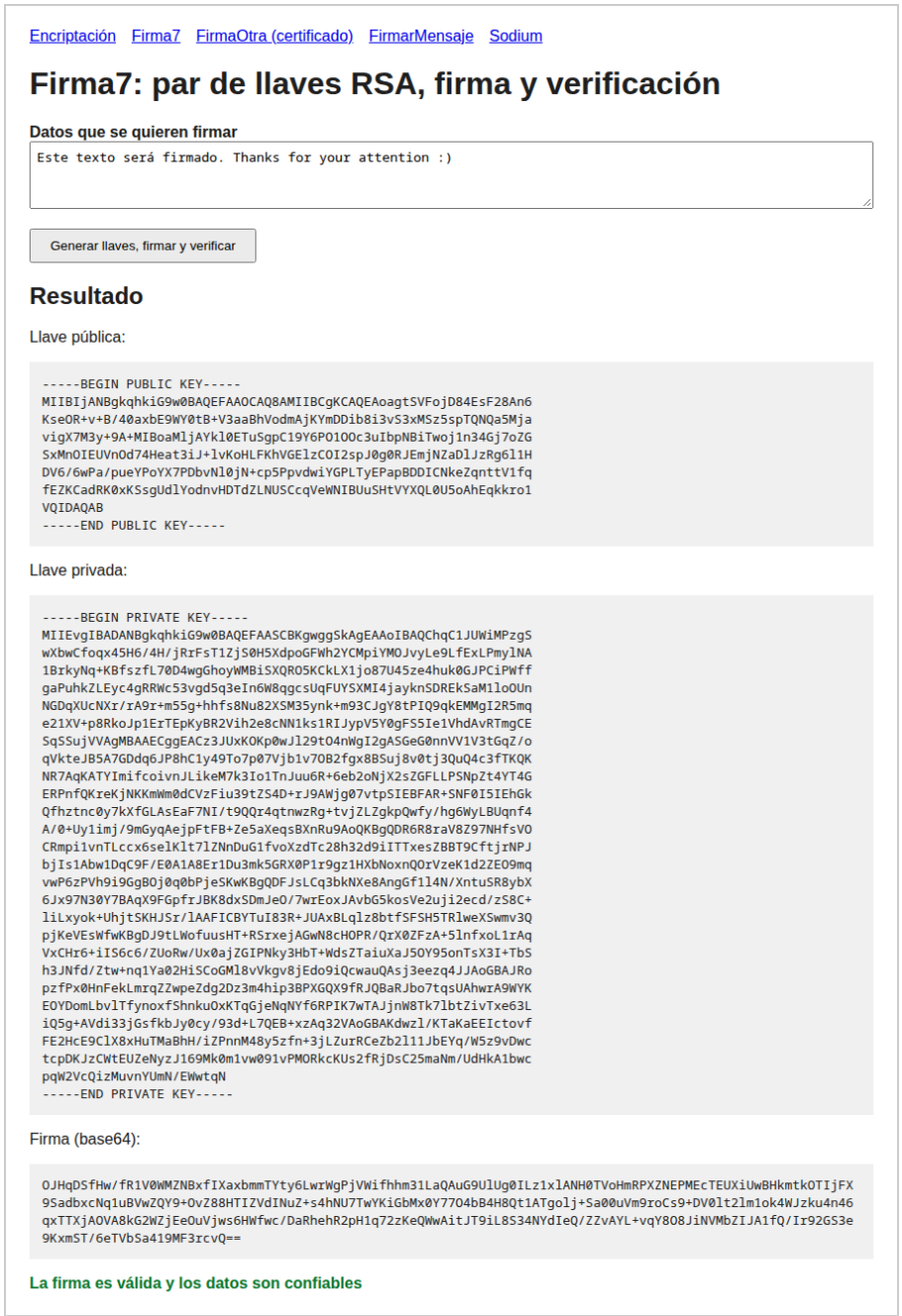


Captura 2. Texto cifrado (IV + datos en base64) y texto descifrado.

Notamos que el IV cambia cada vez que enviamos el formulario, así que el mismo texto con la misma clave da un resultado distinto cada vez. Eso es lo correcto, porque si el cifrado fuera siempre igual alguien podría reconocer patrones. Guardo el IV junto al cifrado porque no es secreto, pero sin él no se puede descifrar.

3.3 Firma7: llaves RSA, firma y verificación

Este script genera un par de llaves RSA de 2048 bits, firma un texto con la llave privada usando SHA-256 y verifica la firma con la llave pública. Además guarda las llaves y la firma en la carpeta `keys/`.

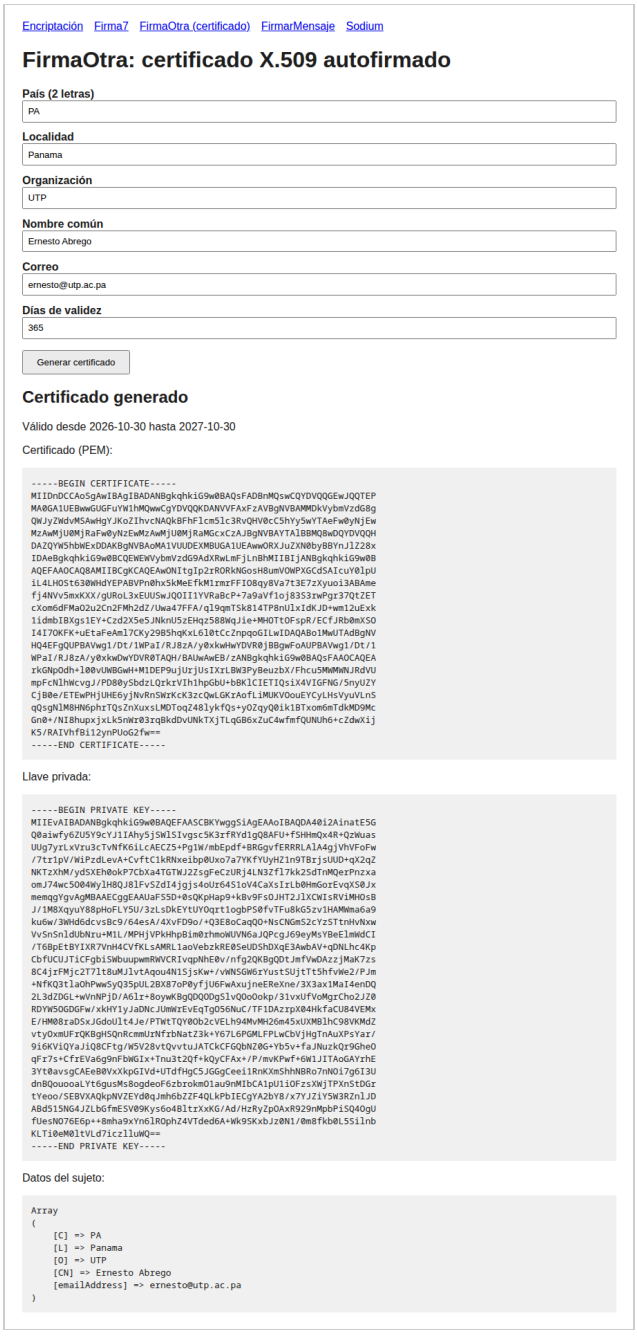


Captura 3. Llaves generadas, firma en base64 y verificación exitosa.

El mensaje final dice que la firma es válida y los datos son confiables. Esto demuestra que el texto es el mismo que se firmó y que la firma salió de la llave privada que corresponde a esa llave pública.

3.4 FirmaOtra: certificado X.509 autofirmado

Aquí llenamos un formulario con nuestros datos (país, localidad, organización, nombre y correo). El script crea una llave, una solicitud de certificado (CSR) y la firma con la misma llave, lo que da un certificado autofirmado válido por 365 días. Guarda el certificado y la llave privada en `keysCert/`.



Captura 4. Certificado X.509 en formato PEM, llave privada y datos del sujeto.

Es autofirmado porque nosotros mismos lo firmamos y no una autoridad certificadora, por eso sirve para pruebas pero un navegador no lo aceptaría como confiable.

3.5 FirmarMensaje: firmar y verificar con el certificado

Este script usa lo que generó el anterior. Firma un mensaje con la llave privada de `keysCert/` y lo verifica con la llave pública que viene dentro del certificado. Le agregamos un segundo campo para poder cambiar el mensaje antes de verificar y ver qué pasa.



Captura 5. Mensaje igual al firmado: la firma es válida.



Captura 6. Mensaje con un cambio pequeño: la firma es inválida.

Con solo agregar unas palabras al mensaje, la firma dejó de ser válida. Eso es la integridad: cualquier cambio, por pequeño que sea, se detecta.

3.6 Dificultades y cómo las resolví

- **Ruta de openssl.cnf:** la guía menciona que en Windows a algunos no les funcionó la ruta de Program Files. Usamos la solución 1: quitamos la ruta fija. En `config.php` dejamos además un bloque que busca el archivo en las rutas comunes de XAMPP, que es la solución 2.
- **Error "invalid digest":** al crear el certificado con PHP 8.5 y OpenSSL 3.5, `openssl_csr_new` fallaba. Lo resolvimos agregando `'digest_alg' => 'sha256'` en los parámetros.
- **Error del generador aleatorio:** el certificado también fallaba con "random number generator: Cannot open file", porque no existía el archivo de aleatoriedad. Lo resolvimos creando uno temporal desde `config.php` y apuntando a él con la variable `RANDFILE`.

4. Documentación técnica de funciones

Estas son las funciones que usamos, organizadas por lo que hacen. En todas la parte criptográfica la hace OpenSSL por dentro, PHP solo expone las funciones.

4.1 Llaves y firma digital (Firma7, FirmarMensaje)

Función	Propósito	Parámetros	Valor devuelto	Proceso criptográfico
`openssl_pkey_new`	Crear un par de llaves	Arreglo con `private_key_bits` (2048), `private_key_type` (RSA), `digest_alg`	Recurso de llave o `false`	Genera dos números primos grandes y a partir de ellos la llave pública y la privada (RSA).
`openssl_pkey_get_details`	Obtener la llave pública y datos de la llave	Recurso de llave	Arreglo; `key` es la pública en PEM	Solo lee los datos, no calcula nada.
`openssl_pkey_export`	Sacar la llave privada como texto PEM	Llave, variable por referencia, contraseña opcional, configuración	`true` o `false`	Codifica la llave en formato PEM (base64) para guardarla en un archivo.
`openssl_get_privatekey` / `openssl_pkey_get_private`	Cargar una llave privada desde texto PEM	Texto PEM	Recurso de llave o `false`	Convierte el texto de nuevo en una llave utilizable.
`openssl_sign`	Firmar datos	Datos, variable de salida por referencia, llave privada, algoritmo (`OPENSSL_ALGO_SHA256`)	`true` o `false`	Calcula el hash SHA-256 de los datos y lo cifra con la llave privada RSA. El resultado es la firma.
`openssl_verify`	Verificar una firma	Datos, firma binaria, llave pública, algoritmo	1 válida, `0` inválida, `-1` error	Descifra la firma con la llave pública, calcula el hash de los datos y compara los dos. Si son iguales, la firma es válida.
`openssl_pkey_get_public`	Obtener la llave pública desde un certificado o PEM	Certificado o llave pública en PEM	Recurso de llave o `false`	Extrae la llave pública para poder verificar.

4.2 Certificados (FirmaOtra)

Función	Propósito	Parámetros	Valor devuelto	Proceso criptográfico
`openssl_csr_new`	Crear la solicitud de certificado (CSR)	Datos del sujeto (país, organización, nombre, correo), llave, configuración	Recurso CSR o `false`	Junta los datos del sujeto con la llave pública y firma la solicitud con la privada.
`openssl_csr_sign`	Firmar la solicitud y crear el certificado	CSR, certificado de la CA (`null` = autofirmado), llave privada, días de validez, configuración	Recurso de certificado o `false`	Firma el certificado con la llave privada. Al ser autofirmado, el emisor y el dueño son el mismo.
`openssl_x509_export`	Exportar el certificado a PEM	Certificado, variable de salida por referencia	`true` o `false`	Codifica el certificado en formato PEM.
`openssl_x509_parse`	Leer los datos del certificado	Certificado	Arreglo con sujeto, fechas, emisor	Solo lee los datos.

4.3 Encriptación simétrica (Encriptacion)

Función	Propósito	Parámetros	Valor devuelto	Proceso criptográfico
`openssl_cipher_iv_length`	Saber cuántos bytes necesita el IV	Nombre del método (`AES-256-CBC`)	Número (16)	Ninguno, solo consulta.
`openssl_random_pseudo_bytes`	Generar el IV aleatorio	Cantidad de bytes	Cadena de bytes aleatorios	Usa el generador aleatorio de OpenSSL.
`openssl_encrypt`	Cifrar un texto	Texto, método, clave, opciones (`OPENSSL_RAW_DATA`), IV	Texto cifrado o `false`	AES-256 en modo CBC: divide el texto en bloques de 16 bytes y cada bloque se mezcla con el anterior antes de cifrarlo.
`openssl_decrypt`	Descifrar un texto	Texto cifrado, método, clave, opciones, el mismo IV	Texto original o `false`	Hace el proceso inverso. Con una clave o IV incorrectos no devuelve el texto.

5. Investigación y ejemplo con Libsodium (Sodium)

5.1 Investigación

Libsodium es una librería de criptografía moderna, y PHP la incluye de forma nativa desde la versión 7.2 como la extensión `sodium`. A diferencia de OpenSSL, que es muy amplia y pensada para protocolos de red y certificados (SSL/TLS), Libsodium busca que sea difícil usar mal la criptografía: elige por uno los algoritmos, maneja bien la aleatoriedad y los nonces, y tiene pocas opciones para equivocarse.

Sus algoritmos principales son:

- **Cifrado simétrico autenticado:** `sodium_crypto_secretbox` usa XSalsa20 para cifrar y Poly1305 para autenticar, así que cifra y comprueba que nadie cambió el dato en una sola llamada.
- **Cifrado asimétrico:** `sodium_crypto_box` usa X25519 para acordar una clave entre dos personas y luego cifra con la misma construcción.
- **Firmas digitales:** `sodium_crypto_sign` usa Ed25519, que es rápido y con llaves pequeñas (32 bytes).
- **Hash de contraseñas:** `sodium_crypto_pwhash` usa Argon2, el algoritmo que recomienda OWASP porque gasta mucha memoria y frena los ataques de fuerza bruta con GPU.
- **Hash general:** `sodium_crypto_generichash` usa BLAKE2b.

Las ventajas que encontré son: es más rápida y sencilla que OpenSSL para cifrar datos, resiste mejor los errores del programador, ya viene incluida en PHP y no necesita configurar el archivo `openssl.cnf`, que fue justo el problema que tuvieron varios compañeros.

Característica	OpenSSL	Libsodium
Propósito principal	Protocolos de red, certificados y criptografía asimétrica (RSA)	Cifrado moderno, firmas y hash de contraseñas
Complejidad	Alta: hay que elegir método, modo, IV, padding	Baja: funciones simples y seguras por defecto
Cifrado de datos	AES-CBC no verifica integridad por sí solo	Cifra y verifica integridad a la vez
Firma digital	RSA (llaves grandes)	Ed25519 (llaves pequeñas, más rápida)
Uso recomendado	HTTPS, certificados X.509	Datos sensibles en la base de datos, contraseñas

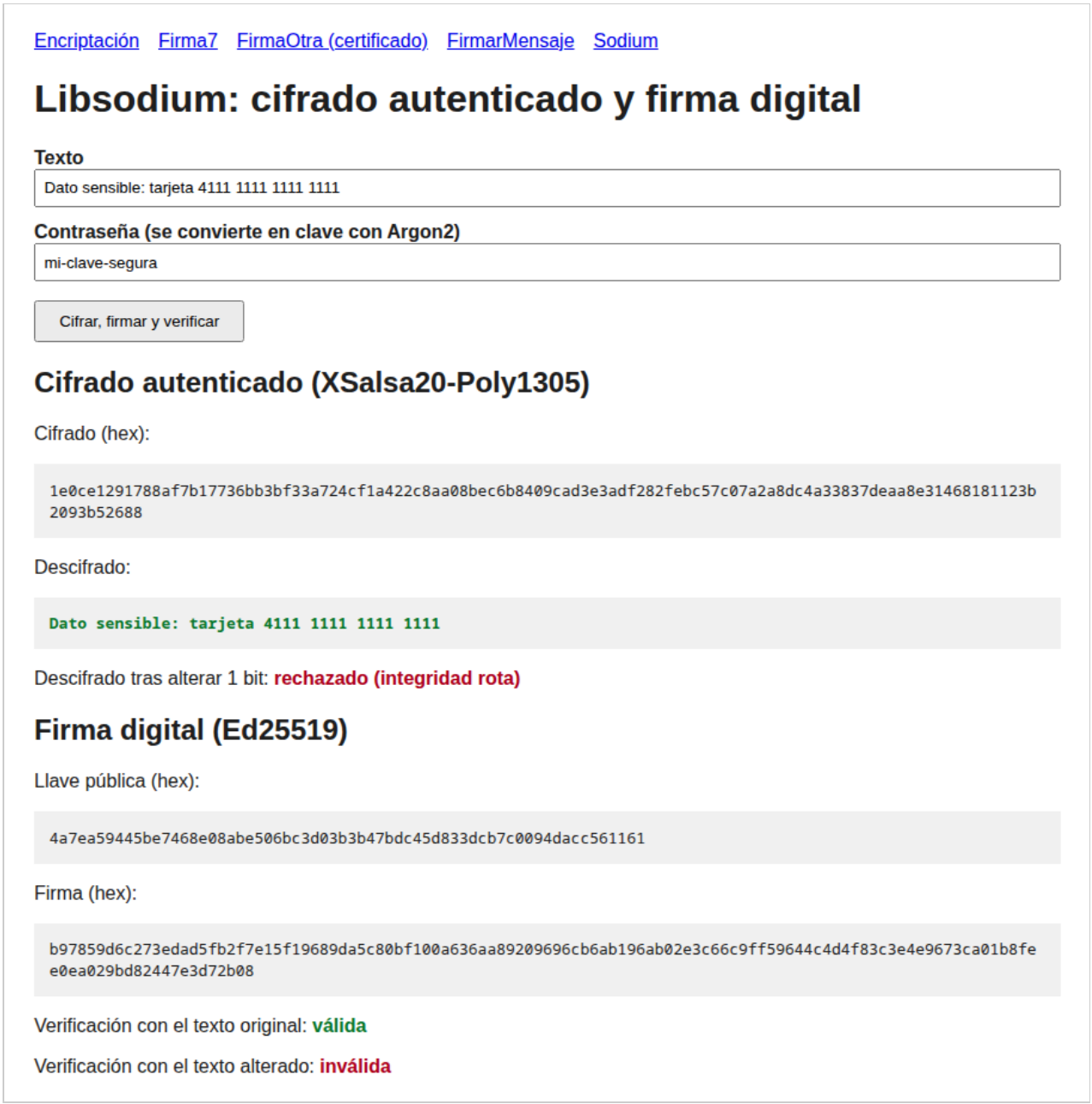
5.2 Ejemplo implementado

Hicimos la página `sodium.php` con un formulario donde escribimos un texto y una contraseña. Hace tres cosas: deriva una clave de la contraseña con Argon2, cifra el texto con `secretbox` y lo firma con Ed25519. Además comprueba qué pasa si alguien altera los datos. Estas son las partes principales del código:

```
// Clave derivada de la contraseña con Argon2
$sal = random_bytes(SODIUM_CRYPTO_PWHASH_SALTBYTES);
$clave = sodium_crypto_pwhash(SODIUM_CRYPTO_SECRETBOX_KEYBYTES, $_POST['clave'], $sal,
    SODIUM_CRYPTO_PWHASH_OPSLIMIT_INTERACTIVE, SODIUM_CRYPTO_PWHASH_MEMLIMIT_INTERACTIVE);

// Cifrado autenticado y descifrado
$nonce   = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$cifrado = sodium_crypto_secretbox($texto, $nonce, $clave);
$claro   = sodium_crypto_secretbox_open($cifrado, $nonce, $clave); // false si fue
alterado

// Firma digital Ed25519
$par     = sodium_crypto_sign_keypair();
$firma   = sodium_crypto_sign_detached($texto, sodium_crypto_sign_secretkey($par));
$ok      = sodium_crypto_sign_verify_detached($firma, $texto,
sodium_crypto_sign_publickey($par));
```



Captura 7. Cifrado autenticado, rechazo del dato alterado y firma Ed25519 verificada.

En la captura se ve que el descifrado devuelve el texto original, y que cuando alteramos un solo bit del cifrado, Sodium lo rechazó y no devolvió nada. Con OpenSSL en modo CBC eso no pasa: descifraría basura sin avisar. En la firma, el texto original sale válido y el texto con una letra de más sale inválido.

6. Conclusiones personales

Con este laboratorio entendimos que proteger datos no es solo “encriptar”: hay que pensar si lo que necesito es que nadie lea el dato (cifrado) o que nadie lo cambie sin que yo me dé cuenta (firma). Las

capturas de FirmarMensaje nos ayudaron a verlo, porque con cambiar unas palabras la firma deja de ser válida.

También nos dimos cuenta de que usar OpenSSL en PHP tiene muchos detalles que se pueden hacer mal, como la ruta del archivo de configuración, el algoritmo de hash o el IV. Nos pasó con el certificado, que falló por el digest y por el generador aleatorio, y tuvimos que revisar los errores uno por uno para resolverlo. Además, nuestro ejemplo de AES-CBC cifra, pero no comprueba que el dato no fue alterado, y la clave la derivamos con un SHA-256 simple, que no es lo ideal para contraseñas. Por eso Libsodium nos pareció la mejor opción para datos sensibles: cifra y verifica en una sola función y usa Argon2 para las contraseñas.

Lo que nos llevamos es que para proteger datos en una aplicación real conviene usar las librerías aprobadas y no inventar nada, como dice la guía, y que OpenSSL sigue siendo la herramienta para lo que Sodium no hace, como los certificados X.509 y HTTPS.

7. Bibliografía

The PHP Group. (s. f.). *OpenSSL*. Manual de PHP. https://www.php.net/manual/es/book.openssl.php

The PHP Group. (s. f.). *Sodium*. Manual de PHP. https://www.php.net/manual/es/book.sodium.php

Denis, F. (s. f.). *Libsodium documentation*. https://doc.libsodium.org

OWASP Foundation. (2021). *A02:2021 – Cryptographic failures*. OWASP Top 10. https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

Josefsson, S., & Liusvaara, I. (2017). *Edwards-curve digital signature algorithm (EdDSA)* (RFC 8032). IETF. https://www.rfc-editor.org/info/rfc8032

Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021). *Argon2 memory-hard function for password hashing and proof-of-work applications* (RFC 9106). IETF. https://www.rfc-editor.org/info/rfc9106

Cooper, D., et al. (2008). *Internet X.509 public key infrastructure certificate and CRL profile* (RFC 5280). IETF. https://www.rfc-editor.org/info/rfc5280

Fong, I. (s. f.). *SeguridadOpenSSL* [Repositorio]. GitHub. https://github.com/Salomon2514/SeguridadOpenSSL